



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Lecture : Multi-Threading



Why should you study this chapter?

- This chapter will help you developing a program in which some tasks executing concurrently.
- Nowadays, in one program, some tasks execute concurrently. You usually saw them in a web page including texts, sounds, images, games, ... changing concurrently.
- Nowadays, operating systems (OS) support many programs running concurrently.
- Open the Task Manager of Windows OS (Ctrl Alt Delete) to see how many programs are running in your computer.

Concepts introduced in the course OOP using Java:

- Object = properties + methods
- Benefits of OO Implementation: encapsulation, inheritance, polymorphism
- Encapsulation is implemented by modifiers
- Inheritance: Subclass inherits declaration of base class (Java is a single inheritance OOP language, the class Object is the ultimate Java class)
- Polymorphism in Java is implemented by overloading and overriding methods

- Class: A template (blueprint) for a group of similar instances
- Interface: the core of some classes
- Abstract class is a class containing abstract methods (some methods are prototyped only)
- Anonymous class: A concrete class developed from an abstract class or an interface but it is not named:
- Nested class: a class which is declared in the declaration of the enclosing class
- Enum type: Declaration of some constants

- **Class Declaration:**

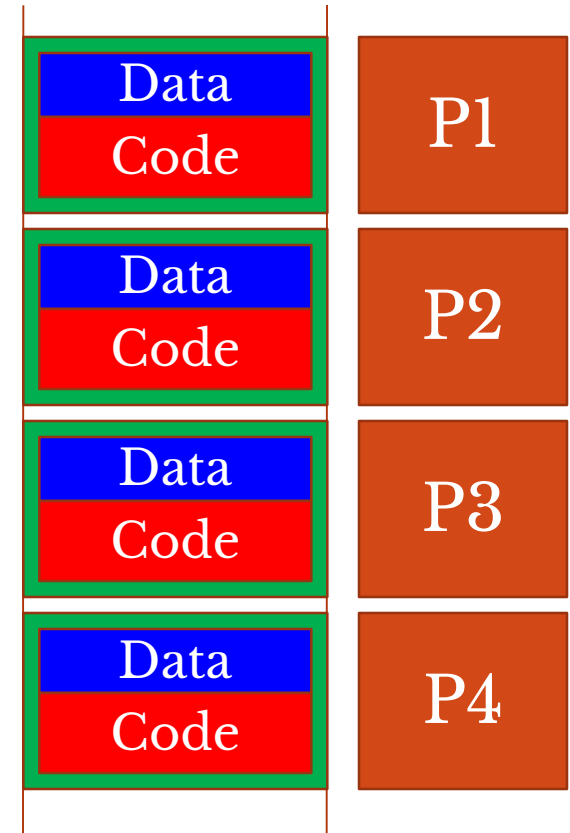
```
[public] class ClassName [extends BaseClass] [implements Interface] {  
    [modifiers] DataType field1 [= initialValue];  
    [modifier] ClassName (params) { // constructor  
        <code>  
    }  
    [modifier] methodName(params) { // method  
        <code>  
    }  
}
```

- **Creating objects:** **ClassName** obj = **new** **ClassName**(params)
- **Accessing an object:** **obj.field** or **obj.methodName(args)**

- Multi-Processing System
- Threads and Multi-Threading Program
- Thread Fundamentals in Java

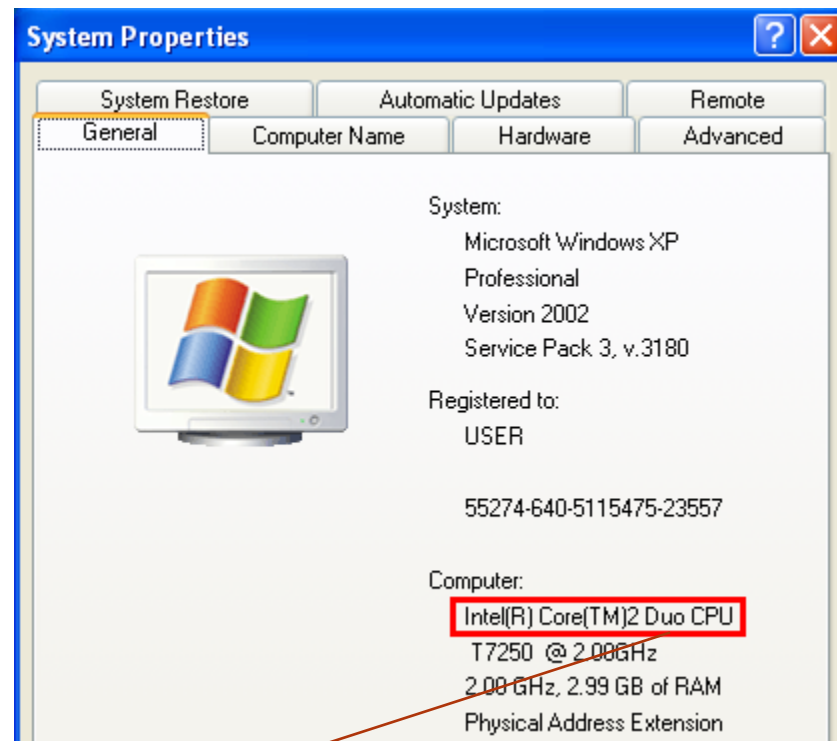
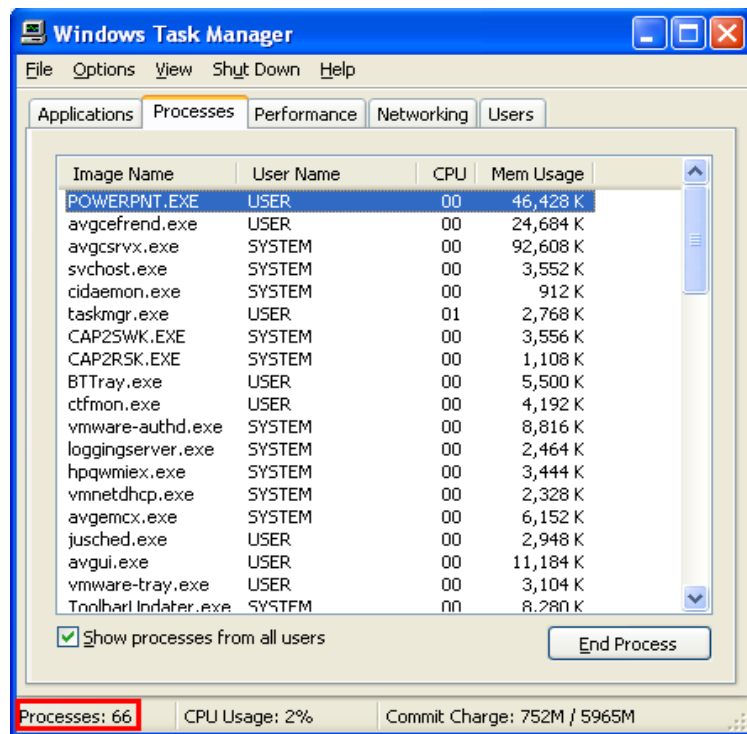
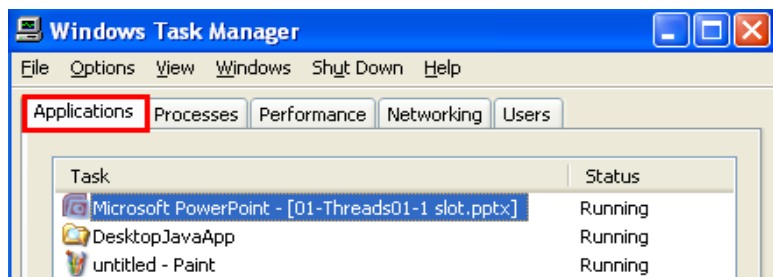
1- Processes and Multi-Processing System

- **Program:** An executable file (data + code) stored in secondary memory(disk).
- **Process:** A program in running. It's data and code are loaded to RAM in a contiguous memory block.
- **Multi-Processing System:** An operating system allows some processes running concurrently



- A **process** has a self-contained execution environment. A process generally has a complete, private set of basic run-time resources; in particular, each process has its own memory space.
- **Multi Processing/ Multi Tasking System**: Almost of operating systems allows many processes executing concurrently.
- Open the **Task Manager** of Windows OS (Ctrl+Alt+Del) to see current processes in your computer.
 - Applications tag contains processes which you start them.
 - Processes tag contains processes which automatically run immediately after the startup of OS.

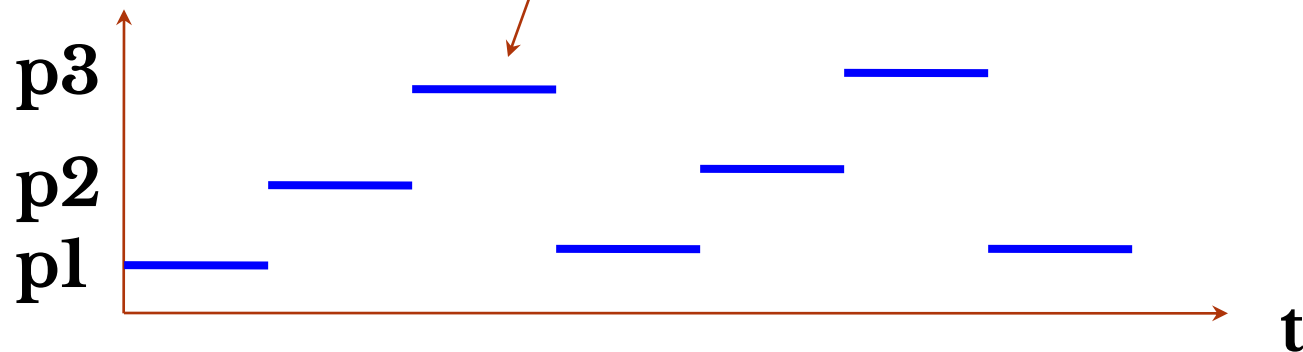
Processes and Multi Processing System...



CPU 2 cores → 1 core runs 33 (66/2) processes
→ Pseudo -parallelism

Processes and Multi Processing System...

- How OS manages processes based on one CPU?
 - Time-sharing mechanism: OS allows each process running in a time slot (time slice, quantum, about 50 to 70 ms). When this duration expires, another process will be chosen to run → The scheduler (a component of OS, trình lập lịch) will choose the current process.

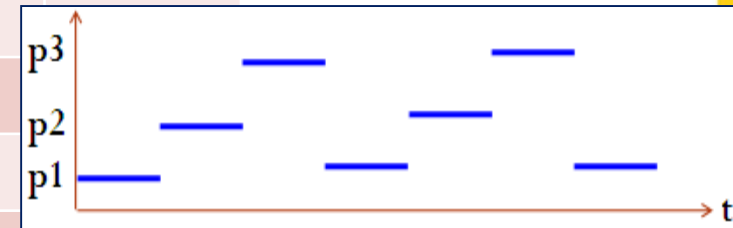


Processes and Multi Processing System...

How can OS manage concurrent processes?

Process Table maintains information of processes.

App	Code Addr	Duration (mili sec)	CPU	...
P1	30320	50	1	
P2	20154	60	2	
P3	10166	70	1	
...	...	...	...	
...	...	...	...	



Time-slicing mechanism. Each process is allocated resources (CPU, ...) for executing in a time-slot (such as 50 milliseconds). When the time duration expires, this process will pause to yield resources to the next process which will be chosen by the scheduler of OS.

Memory

2- Threads and Multi-Threading

- Thread, is sometimes called *lightweight processes*, is a *running code unit*.
- Threads exist within a process — **every process has at least one thread (main thread)**. Threads share the process's resources, including memory and open files. This makes for efficient, but potentially problematic, communication.
- Both processes and threads are provided an execution environment, but creating a new thread requires fewer resources than creating a new process.
- Multithreaded execution is an essential feature of the Java platform. Threads in a program are managed by the JVM. Scheduler in JVM will choose a current thread.

Threads and Multi-Threading ...

How can JVM manage threads

- Thread is a smallest unit code in an application that performs a special job.
- A program can have several threads they can be executed concurrently.

Thread Table maintains information of threads

Thread	Code Addr	Duration (mili sec)	CPU	State
Thread 1	10320	15	1	ready
Thread 2	40154	17	2	ready
Thread 3	80166	22	1	sleep
...	...	...	...	

App

Data

Code of thread1

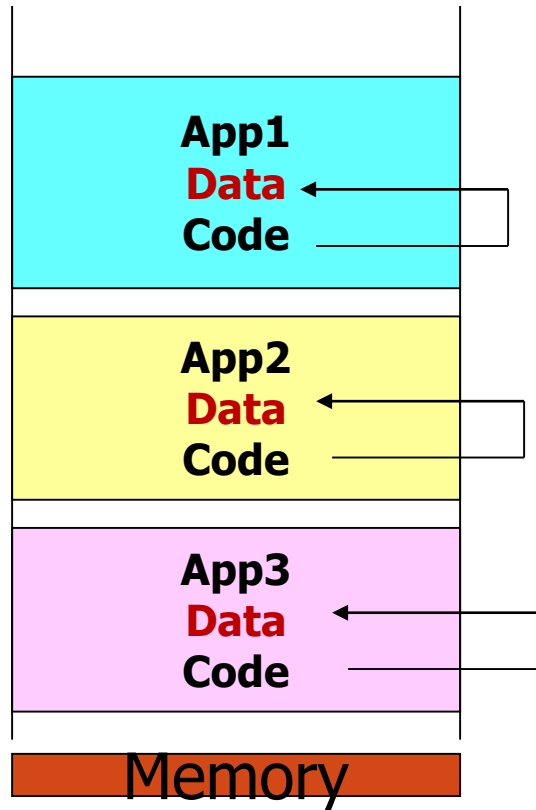
Code of thread2

Code of thread3

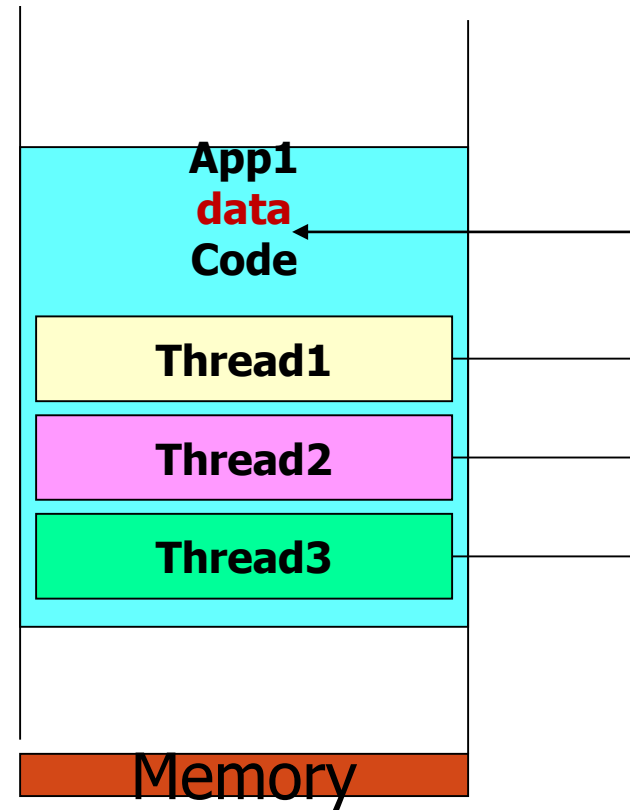
Memory of a multi-thread process

Threads and Multi-Threading ...

Processes VS Threads

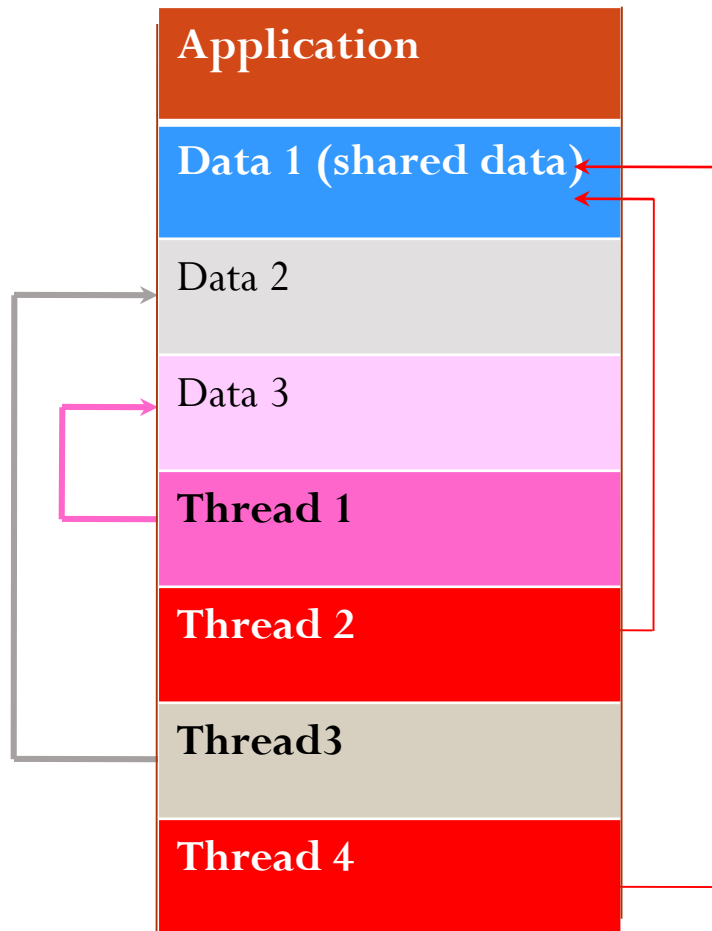


Protection mechanism in OS does not allow this process accessing addresses in others



Threads in a process can access common data of this process

Threads and Multi-Threading ... Race Conditions



Suppose that Thread2 and Thread4 concurrently execute.

Time	Thread 2	Thread4
1	Data1=10	...
2		...
3		Data1= 100
4		...
5	...	Y=?
6	Y= 2*Data1;	
7		...

A race occurs

Need Synchronization

3- Thread Fundamentals in Java

- **Threading supports in Java:**
 - The **java.lang.Thread** class
 - The **java.lang.Object** class
 - The Java language and JVM (Java Virtual Machine)
- **How to create a thread in Java?**
 - (1) Create a subclass of the **java.lang.Thread** class and override the method **run()**
 - (2) Create a class implementing the **Runnable interface** and override the **run()** method.

Create a subclass of the Thread class

```

1  /* Thread1.java */
2  public class Thread1 extends Thread {
3
4      public Thread1() {
5          super ();
6      }
7
8      public void run() {
9          System.out.println("I'm a child thread");
10     }

```

Thread	Code Addr	State	...
main	8000	run	...
t	10320	run	

Every process has at least one thread (**main thread**).

```

1  public class Thread1Using {
2      public static void main(String args[])
3      {
4          Thread1 t= new Thread1();
5          System.out.println("I'm the main thread.");
6          t.start();
7          System.out.println("Hello");
8      }

```

Output - ThreadDemo (run-single)

```

compile:
run-single:
I'm the main thread.
Hello
I'm a child thread

```

?

This process has 2 threads running concurrently. At a time, order of their execution is decided by the scheduler.

The start() method is implemented in the Thread class. This method calls the method run().

Class implements the Runnable interface

```

1 public class RunnableThread implements Runnable
2
3     public RunnableThread() {
4     }
5
6     public void run() {
7         System.out.println("I'm a child thread");
8     }

```

Thread	Code Addr	State	...
main	8000	run	...
t	10320	run	

```

1 public class RunnableClassUsing {
2
3     public static void main(String args[]) {
4         RunnableThread obj= new RunnableThread();
5         Thread t= new Thread (obj);
6         t.start();
7         System.out.println("I'm the main thread.");
8         System.out.println("Hello");
9     }
10 }

```

Output - ThreadDemo (run-single)

```

compile:
run-single:
I'm the main thread.
Hello
I'm a child thread

```

The java.lang.Thread class

Declaration

public class **Thread** extends **Object** implements **Runnable**

set/get-is

Constructor

Thread()

Thread(Runnable target)

Thread(Runnable target, String name)

Thread(String name)

Thread(ThreadGroup group, Runnable target)

Thread(ThreadGroup group, Runnable target, String name)

Thread(ThreadGroup group, Runnable target, String name, long stackSize)

Thread(ThreadGroup group, String name)

Properties

id

name

state

threadGroup

daemon

priority

Common Methods

start()

join ()

sleep (milisec)

yield()

notify()

notifyAll()

wait()

static int	<u>MAX_PRIORITY</u>	10
static int	<u>MIN_PRIORITY</u>	1
static int	<u>NORM_PRIORITY</u>	5

Using some methods of the Thread class

```

ThreadProperties.java x
1 public class ThreadProperties extends Thread{
2     public ThreadProperties(String threadName) {
3         super(threadName);
4         this.start();
5     }
6     public static void showProperties(Thread t){
7         System.out.println("I'm the " + t.getName() + " thread");
8         System.out.println("--My ID:" + t.getId());
9         System.out.println("--My name:" + t.getName());
10        System.out.println("--My priority:" + t.getPriority());
11        System.out.println("--My state:" + t.getState());
12        System.out.println("--I'm a daemon:" + t.isDaemon());
13        System.out.println("--I'm alive:" + t.isAlive());
14    }
15    public void run(){
16        showProperties(this);
17    }
18    public static void main (String args[]){
19        System.out.println("Thread count:" + Thread.activeCount());
20        Thread t= Thread.currentThread();
21        showProperties(t);
22        ThreadProperties t1= new ThreadProperties("Son1");
23        ThreadProperties t2= new ThreadProperties("Son2");
24        System.out.println("Thread count:" + Thread.activeCount());
25    }
26 }
    
```

Output

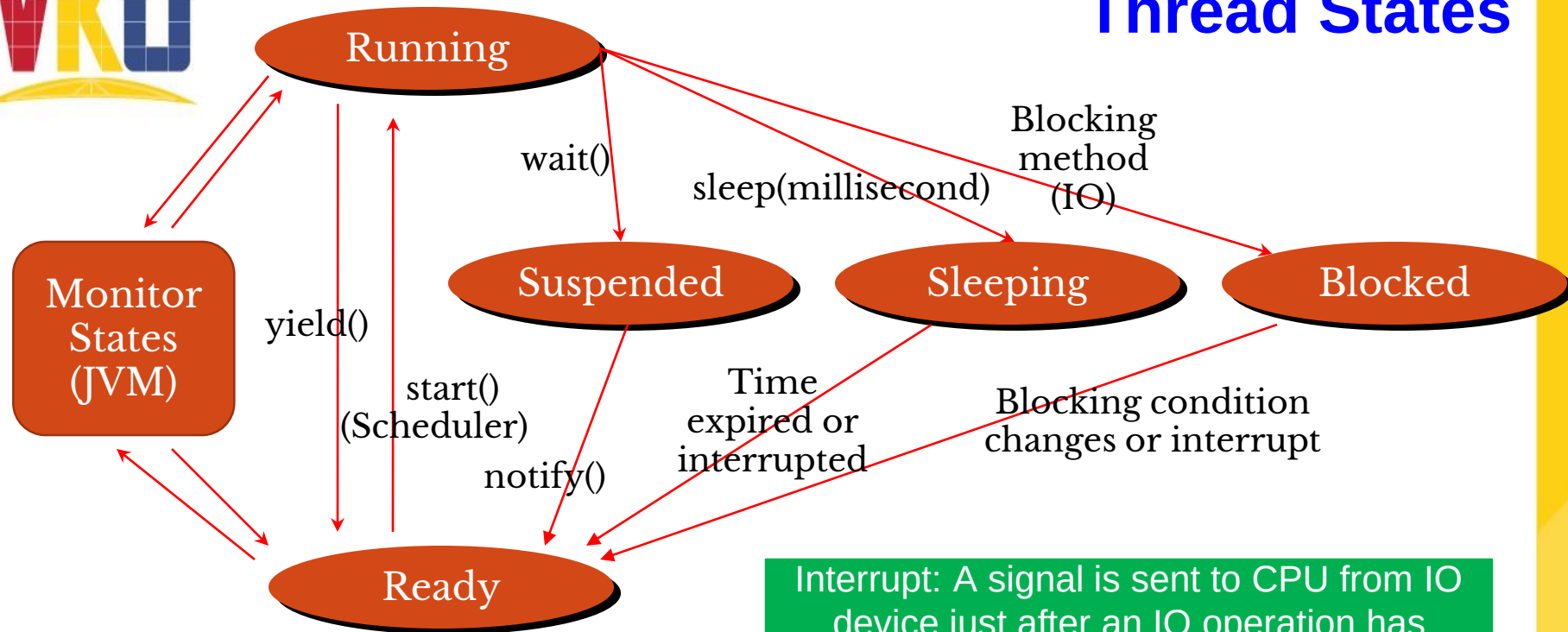
Debugger Console x ThreadC

run-single:

```

Thread count:1
I'm the main thread
--My ID:1
--My name:main
--My priority:5
--My state:RUNNABLE
--I'm a daemon:false
--I'm alive:true
Thread count:3
I'm the Son2 thread
--My ID:9
--My name:Son2
--My priority:5
--My state:RUNNABLE
--I'm a daemon:false
--I'm alive:true
I'm the Son1 thread
--My ID:8
--My name:Son1
--My priority:5
--My state:RUNNABLE
--I'm a daemon:false
--I'm alive:true
    
```

Thread States



Interrupt: A signal is sent to CPU from IO device just after an IO operation has terminated.

Only one of ready threads will be chosen by the JVM scheduler at a time.

Ready: As soon as it is created , it can enter the **running** state when JVM's processor is assigned to it.

Running: It get full attention of JVM's processor which executes the thread's **run()** method

Dead: When the **run()** method terminates.

Concepts were introduced:

- Definitions: Program, Process, Thread
- Multi-processing system
- Multi-threading programming in Java
- Thread Fundamentals in Java
- Thread states

In the previous session:

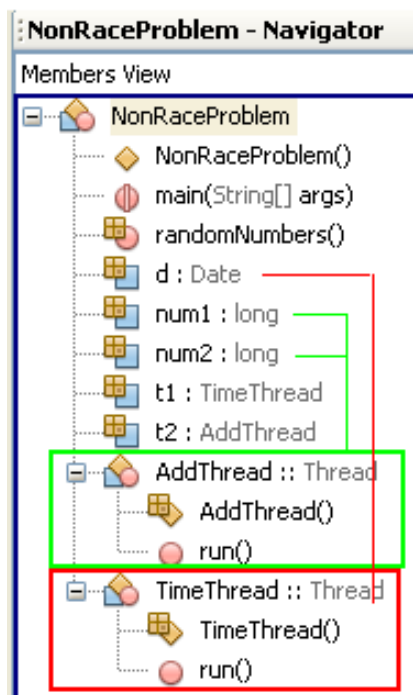
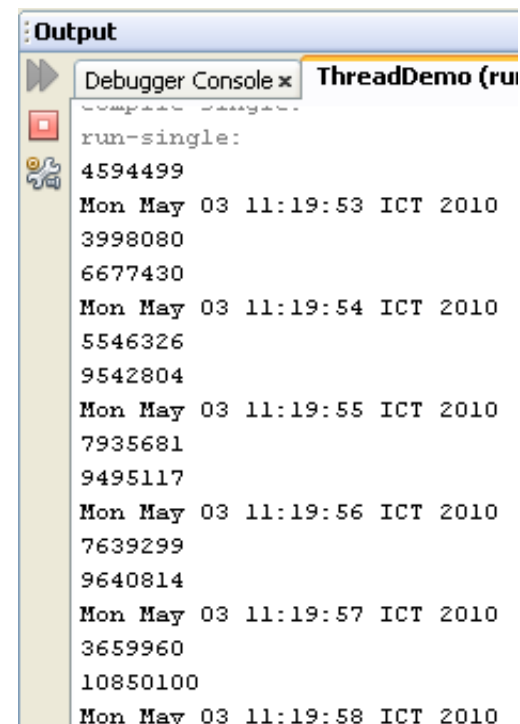
- Definitions: Program, Process, Thread
- Multi-processing system
- Multi-threading programming in Java
- Thread Fundamentals in Java
- Thread states

- How to develop a non-race multi-thread applications
- How to develop a multi-thread applications in which some threads accessing common resources?
- **Contents:**
 - Demonstrations
 - Monitors, Waiting and Notifying
 - Deadlocks

Non-race Demonstration

This program contains 2 threads:

- The first thread that will print out the system time for every second.
- The second will print out the sum of two random integers for every half of a second.

The screenshot shows the 'Output' window with the 'Debugger Console' tab selected. The output text is as follows:

```
run-single:
4594499
Mon May 03 11:19:53 ICT 2010
3998080
6677430
Mon May 03 11:19:54 ICT 2010
5546326
9542804
Mon May 03 11:19:55 ICT 2010
7935681
9495117
Mon May 03 11:19:56 ICT 2010
7639299
9640814
Mon May 03 11:19:57 ICT 2010
3659960
10850100
Mon May 03 11:19:58 ICT 2010
```

Non-race Demonstration...

```

1 import java.util.Date;
2 public class NonRaceProblem {
3     Date d=null;
4     long num1=0, num2=0;
5     // 2 threads of inner Threads, declared below
6     TimeThread t1 = new TimeThread ();
7     AddThread t2 = new AddThread ();
8
9     public NonRaceProblem() {
10         d=new Date(System.currentTimeMillis());
11         randomNumbers(); t1.start(); t2.start();
12     }
13     void randomNumbers() {
14         num1= Math.round(Math.random()*100000000);
15         num2= Math.round(Math.random()*100000000);
16     }
17
18     public static void main(String args[]) {
19         NonRaceProblem obj= new NonRaceProblem();
20     }
21 }

```

Output

Debugger Console x ThreadDemo (r)

run-single:

```

4594499
Mon May 03 11:19:53 ICT 2010
3998080
6677430
Mon May 03 11:19:54 ICT 2010
5546326
9542804
Mon May 03 11:19:55 ICT 2010

```

```

17 // Inner class 1: Thread for printing out the time
18 class TimeThread extends Thread{
19     TimeThread() { super(); }
20     public void run() {
21         while (true) {
22             try {
23                 System.out.println(d);
24                 this.sleep(1000);
25                 d=new Date(System.currentTimeMillis());
26             }
27             catch(java.lang.InterruptedException e){
28                 e.printStackTrace();
29             }
30         }
31     }
32 }

```

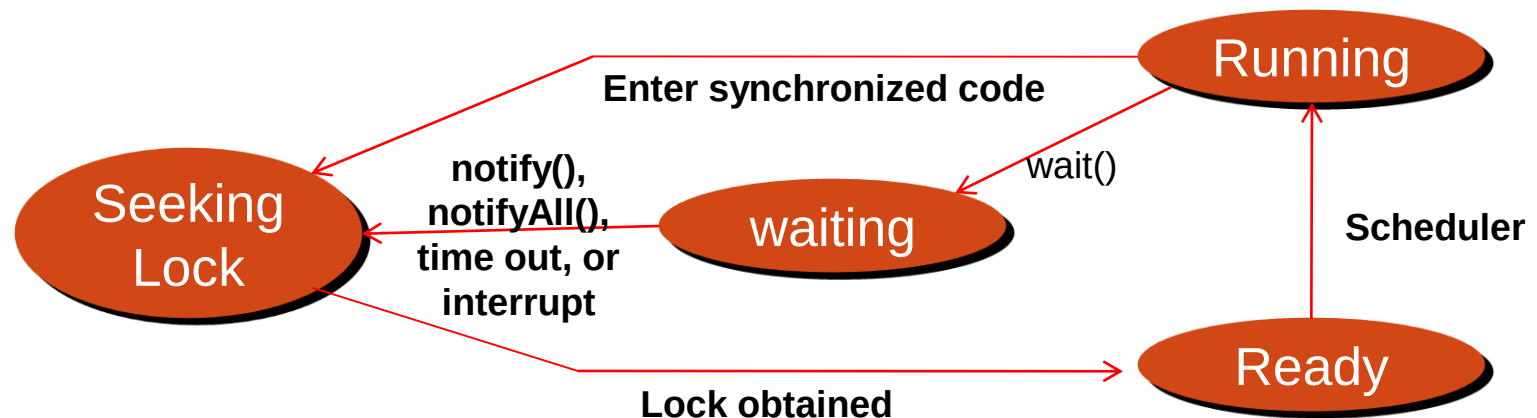
```

33 // Inner class 2: Thread for printing out sum of 2 numbers
34 class AddThread extends Thread{
35     AddThread() { super(); }
36     public void run() {
37         while (true){
38             try { System.out.println(num1+ num2);
39                 randomNumbers();
40                 this.sleep(500);
41             }
42             catch(java.lang.InterruptedException e){
43                 e.printStackTrace();
44             }
45         }
46     }
47 }

```

4- Monitors, Waiting and Notifying

- Some threads can access common resources concurrently. We must synchronize accessing common resources → Every object has a *lock*
- *Lock*: an extra variable is added by the compiler for monitoring the state of common resource. Before a thread accesses the common resource, the lock is tested.
- After a thread has the lock (it is permitted to access the common resource), it can access common resource. When it did, it needs notifying to others thread (wait-notify mechanism).



Monitors, Waiting and Notifying...

- *Two ways to mark code as synchronize*

- Synchronize an entire method: *Let the **synchronized** modifier in the method's declaration.*

```
synchronized Type Method(args) {
    <code>
}
```

A class contains synchronized code is called monitor.

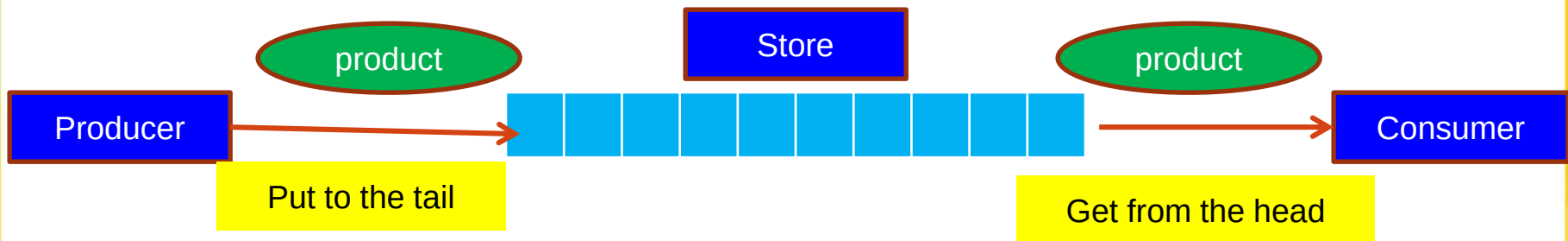
- Synchronize some method of an object

```
Type Method ( args) {
    .....
    synchronized ( object_var) {
        object_var.method1(args);
        object_var.method2(args);
    }
}
```

Monitor: A technique to encapsulate common resources to a class. When a thread wants to access common resources, it must call a public synchronized method. As a result, common resources are accessed in successive manner.

Demo: The Producer-Consumer Problem

- Producer makes a product then puts it to a store.
- Consumer buys a product from a store.
- Selling Procedure: First In First Out



Attention!:

Store is common resource of 2 threads: producer and consumer.

→ Store is a monitor and its activities needs synchronization

Synchronizing:

- * After a thread accessed common resource, it should sleep a moment or it must notify to the next thread (or all thread) in the thread-pool to awake and execute.
- * Use the **synchronized** keyword to declare a method that will access common resource.

The Producer-Consumer Problem

```
public class Store {
    int maxN=50; // maximum number of products can be contains int the store
    long [] a; // product list
    int n; // current number of products
    public Store() { n=0; a=new long[maxN]; }
    private boolean empty() { return n==0; }
    private boolean full() { return n==maxN; }
    public /* synchronized */ boolean put(long x) {
        if (full()) return false;
        a[n++]=x;
        try { Thread.sleep(500); }
        catch (Exception e) { }
        return true;
    }
    public /* synchronized */ long get() {
        long result=0;
        if (!empty()) {
            result=a[0]; // get the product at the front of line
            for (int i=0;i<n-1;i++) a[i]=a[i+1]; // shift products up.
            n--;
        }
        try { Thread.sleep(500); }
        catch (Exception e) { }
        return result;
    }
}
```

A product is simulated as a number.

/* synchronized */
No synchronization

The Producer-Consumer Problem

```
public class Producer extends Thread{
    Store store=null;
    long index=1;    // index of product that will be made
    public Producer(Store s) {
        store=s;
    }
    public void run(){
        while (true){
            try {
                boolean result= store.put(index);
                if (result==true) System.out.println("** Product " + (index++)+ " is made.");
                else System.out.println("Store is full!");
            }
            catch (Exception e) {
            }
        }
    }
}
```

The Producer-Consumer Problem

```
public class Consumer extends Thread {
    Store store=null;
    public Consumer(Store s) {
        store=s;
    }
    public void run(){
        while (true){
            try {
                long x= store.get();
                if (x>0) System.out.println("-- Product " + x + " is bought.");
                else System.out.println("Consumer is waiting for new product.");
            }
            catch (Exception e) {
            }
        }
    }
}
```

```
public class ProducerConsumerProblem {
    Store store;
    Producer pro;
    Consumer con;
    public ProducerConsumerProblem() {
        store= new Store(); pro= new Producer(store); con= new Consumer(store);
        pro.start(); con.start();
    }
    public static void main (String args[]){
        ProducerConsumerProblem obj=new ProducerConsumerProblem();
    }
}
```


The Producer-Consumer Problem

Synchronization is not used

Synchronization is used:

Output - ThreadDemo (run-single) #2

```

>> compile:
run-single:
** Product 1 is made.
-- Product 1 is bought.
-- Product 2 is bought.
** Product 2 is made.
Consumer is waiting for new product.
** Product 3 is made.
** Product 4 is made.
-- Product 3 is bought.
** Product 5 is made.
-- Product 4 is bought.
    
```

Output - ThreadDemo (run-single)

```

>> run-single:
** Product 1 is made.
** Product 2 is made.
-- Product 1 is bought.
-- Product 2 is bought.
Consumer is waiting for new product.
Consumer is waiting for new product.
Consumer is waiting for new product.
Consumer is waiting for new product.
** Product 3 is made.
** Product 4 is made.
-- Product 3 is bought.
-- Product 4 is bought.
Consumer is waiting for new product.
Consumer is waiting for new product.
    
```

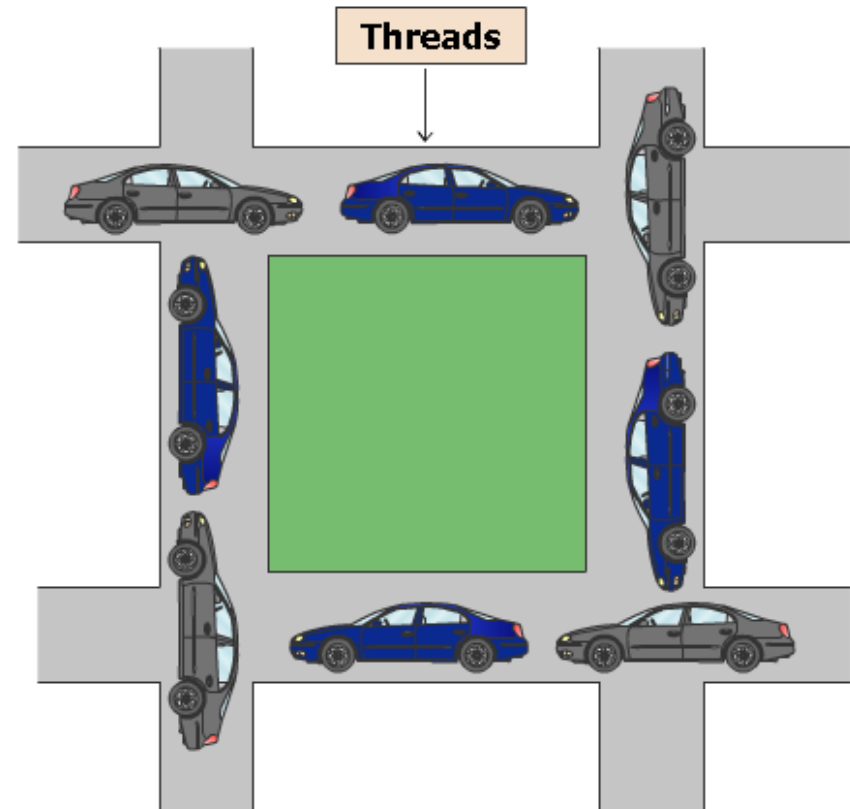
5 - Deadlock

What is deadlock?

Deadlock describes a situation where two or more threads are blocked forever, waiting for each other → All threads in a group halt.

When does deadlock occur?

There exists a circular wait the lock that is held by other thread.



Nothing can ensure that DEADLOCK do not occur.

Deadlock Demo.

```
public class DeadLockDemo implements Runnable {
    DeadLockDemo assistance=null; // giám đốc có trợ lý
    int a=100, b=200;
    public synchronized void changeValues() {
        try{ Thread.sleep(500); a++; b++; }
        catch(Exception e) { }
    }
    public synchronized void run(){
        while (true)
        { try { System.out.println(Thread.currentThread().getName());
            System.out.println("a=" + a);
            System.out.println("b=" + b);
            Thread.sleep(500);
        }
        catch(Exception e) { }
        assistance.changeValues();
    }
}
```

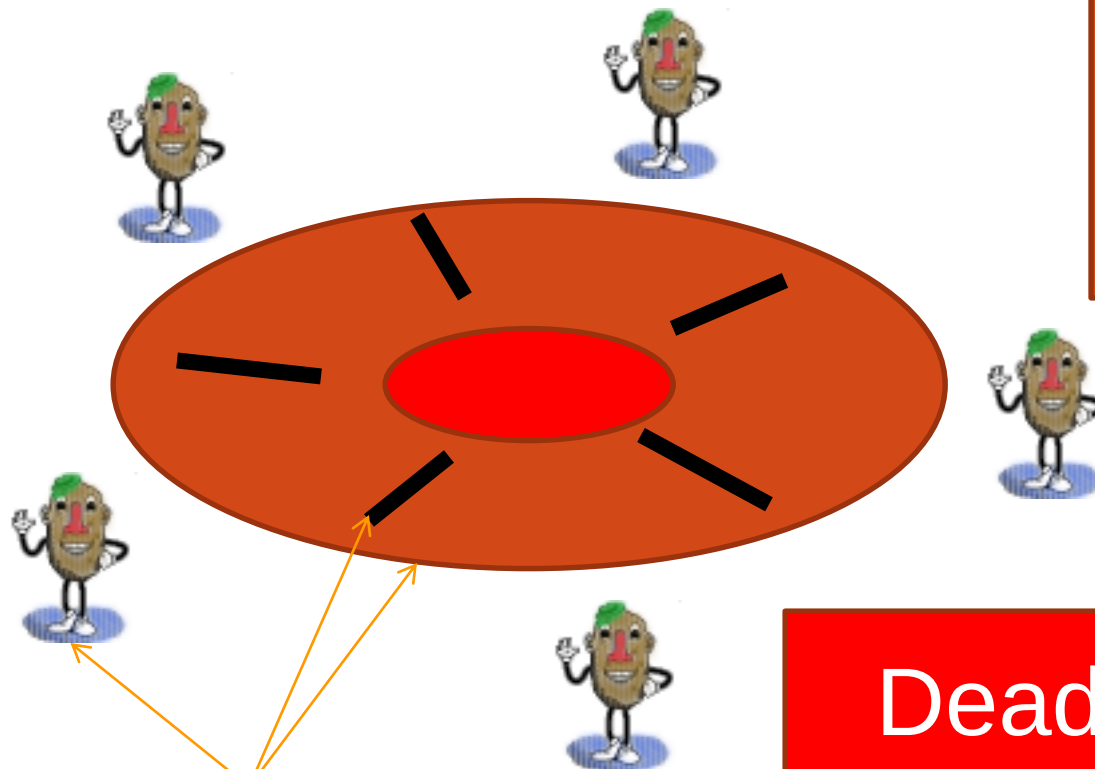
Output - ThreadDemo (run-single) #2

```
init:
deps-jar:
compile-single:
run-single:
Thread-2
a=100
b=200
Thread-1
a=100
b=200
```

```
public static void main(String args[]) {
    DeadLockDemo person1= new DeadLockDemo();
    DeadLockDemo person2= new DeadLockDemo();
    person1.assistance= person2; // hai giám đốc
    person2.assistance= person1; // lại là trợ lý của nhau
    Thread t1= new Thread(person1,"Thread-1");
    Thread t2= new Thread(person2,"Thread-2");
    t1.start();
    t2.start();
    try {
        t1.join();// t1 will be executed to the end
        t2.join();// t2 will be executed to the end
    }
    catch(Exception e) { }
}
```

The Philosophers Problem

Wait-Notify
Mechanism, a way
helps preventing
deadlocks



Deadlock!

3 classes

The Philosophers Problem

```
package threadpkg;
public class ChopStick {
    boolean ready;
    ChopStick(){
        ready=true;
    }
    public synchronized void getUp()
    { while (!ready)
      { try {
          System.out.println("A philosopher is waiting for a chopstick.");
          wait();
        }
        catch (InterruptedException e){
            System.out.println ("An error occurred!");
        }
      }
      ready=false;
    }
    public synchronized void getDown ()
    { ready= true;
      notify();
    }
}
```

Thread table

Thread	Code Addr	Duration (mili sec)	CP U	State
Thread 1	10320	15	1	Suspended → Ready
Thread 2	40154	17	2	Suspended
Thread 3	80166	22	1	Suspended
...	...	...	...	

The Philosophers Problem

```
package threadpkg;

public class Philosopher extends Thread{
    ChopStick leftStick, rightStick; // He/she needs 2 chop sticks
    int position; // His/her position at the dinner table
    Philosopher(int pos, ChopStick lStick, ChopStick rStick)
    { position=pos ; leftStick=lStick; rightStick=rStick;
    }
    public void eat()
    { leftStick.getUp(); rightStick.getUp();
      System.out.println("The " + position + "(th) philosopher is eating");
    }
    public void think()
    { leftStick.getDown(); rightStick.getDown();
      System.out.println("The " + position + "(th) philosopher is thinking\\");
    }
}
```

The Philosophers Problem



```
public void run()
{ while (true)
  { eat();
    try { sleep(1000); }
    catch (InterruptedException e)
    { System.out.println("An error occurred!");
    }
    think();
    try { sleep(1000); }
    catch (InterruptedException e)
    { System.out.println("An error occurred!");
    }
  }
}
```

The Philosophers Problem

```
package threadpkg;
public class DinnerTable {
    static int n;
    static ChopStick[] sticks = new ChopStick[5];
    static Philosopher[] philosophers = new Philosopher[5];

    public static void main (String args[])
    {
        n=5;
        int i;
        for (i=0;i<n;++i) sticks[i]=new ChopStick();
        for (i=0;i<n;++i) philosophers[i] =
            new Philosopher (i,sticks[i],sticks[(i+1)%5]);
        for (i=0;i<n;++i) philosophers[i].start();
    }
}
```

Output - DJA_P1 (run)

```

The 0(th) philosopher is eating
The 1(th) philosopher is thinking"
The 3(th) philosopher is thinking"
The 2(th) philosopher is eating
A philosopher is waiting for a chopstick.
The 0(th) philosopher is thinking"
A philosopher is waiting for a chopstick.
The 4(th) philosopher is eating
The 2(th) philosopher is thinking"
A philosopher is waiting for a chopstick.
The 1(th) philosopher is eating
```


Concepts were introduced:

- Definitions: Program, Process, Thread
- Multi-processing system
- Multi-threading programming in Java
- Thread Fundamentals in Java
- Synchronizing access to common resource.
- Monitoring thread states: Wait-notify mechanism.
- If you want some tasks executing concurrently, multi-threading is a solution.

Workshop 1 (with report): The producer-consumer problem and

Workshop 2 (with report): the philosophers problem.



Thank You